Computer Science C73
Scarborough Campus

September 25, 2024
University of Toronto

Solutions for Homework Assignment #3

Answer to Question 1.

a. The basic reasoning is as follows: Let A_1, A_2, and A_3 be the first third, middle third, and last third of A, respectively. After the first recursive call (which sorts A_1 and A_2) A_2 contains elements that are greater than or equal to the elements in A_1. Thus, after the second recursive call (which sorts A_2 and A_3), A_3 contains elements that are greater than or equal to the elements in A_2 and therefore also in A_1 — i.e., it contains the largest one-third of the elements of A in sorted order. The third recursive call sorts the remaining elements in A_1 and A_2, all of which are smaller than the elements in A_3.

We can make this argument more rigorous by using complete induction on the length n of $A[\ell..r]$, i.e. $n = r - \ell + 1$, to prove that the algorithm correctly sorts subarrays of length n, for every $n \geq 1$. (As suggested, you may assume for simplicity that n is a power of 3, though we will not do so in this proof.)

Suppose the algorithm sorts $A[\ell..r]$ when the length of $A[\ell..r]$ is strictly less than n. We will prove that it also sorts $A[\ell..r]$ when the length of $A[\ell..r]$ is equal to n.

If $n = 1$ or $n = 2$ this is immediate from lines 1–2 or lines 3–4, respectively.

If $n \geq 3$, let $t = \lceil 2(r - \ell + 1)/3 \rceil$, $A_1 = A[\ell..r - t]$ (the first third of A), $A_2 = A[r - t + 1..\ell + t - 1]$ (the middle third of A), and $A_3 = A[\ell + t..r]$ (the final third of A). Also, denote by A^1, A^2, and A^3 the content of A after the first, second, and third recursive call respectively (line 8, 9, and 10). So A_i^j denotes the content of the i-th third of A after the j-th recursive call.

It is easy to see that the lengths of the subarrays $A[\ell..\ell + t - 1]$ and $A[r - t + 1..r]$ are strictly less than n, and so by the induction hypothesis, the three recursive calls correctly sort the relevant subarrays.

By the properties of sorting, it is obvious that:

$$\text{for all } x \text{ in } A_2^1 \text{ and all } y \text{ in } A_1^1,\ x \geq y \tag{1}$$

$$\text{for all } x \text{ in } A_3^2 \text{ and all } y \text{ in } A_2^2,\ x \geq y \tag{2}$$

Now we show that

$$\text{for all } x \text{ in } A_3^2 \text{ and all } y \text{ in } A_1^2,\ x \geq y \tag{3}$$

To see this, let x be in A_3^2 and y be in A_1^2. Since the second recursive call does not affect the first third of A, y is also in A_1^1. There are two cases:

Case 1. No z in A_2^1 is in A_2^2. (In other words, the sorting done by the second recursive call caused every element in the middle third of A to move to the last third.) Therefore, x is one of the values that were in A_2^1. Thus, by (1), $x \geq y$.

Case 2. Some z in A_2^1 is in A_2^2. (In other words, the sorting done by the second recursive call caused some element in the middle third of A to remain there.) By (2), $x \geq z$ (since z is in A_2^2). By (1), $z \geq y$ (since z is also in A_2^1). Therefore $x \geq y$.

By (2) and (3), after the second recursive call, A_3 contains the largest third of the elements of A, in sorted order. Thus, the third recursive call, which does not affect A_3, sorts the remaining two-thirds of the elements of A in A_1 and A_2. Thus, after the third recursive call all of A is sorted.

b. The recurrence equation that describes the running time of LoopySort(A, ℓ, r) when $r - \ell + 1 = n$ is $T(n) = 3T(2n/3) + c$, since there are three recursive calls, each on input 2/3 the size of the original, and a constant amount additional work. In terms of the parameters of the Master Theorem, we have $a = 3$, $b = 3/2$, and $d = 0$. Since $a > b^d$, the third case of the Master Theorem applies, and we have $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{2.71})$ ($\log_b a = \log_{1.5} 3 = \ln 3 / \ln 1.5 \approx 2.71$).

c. The running time of Bubblesort is $\Theta(n^2)$. So, LoopySort is much worse than even the simple quadratic sorting algorithms such as Bubblesort — let alone the $\Theta(n \log n)$ ones such as Mergesort or Heapsort. Bad idea!

Answer to Question 2. Note that A has $n = 2(k-1)+1 = 2k-1$ positions, which is an odd number.

Algorithm: Informally, the idea is as follows: If $n = 1$, then clearly the singleton is position 1. Otherwise, we consider the middle position m of A and its successor $m+1$. If $A[m] = A[m+1]$ then these two successive positions form a couple; it turns out (and we will prove later — see Claim 1 below) that the singleton is in whichever of $A[1..m-1]$ or $A[m+2..n]$ has an odd number of positions (only one of the two can, since $A[1..n]$ has an odd number of positions). If, on the other hand, $A[m] \neq A[m+1]$, it turns out (see Claim 2 below) that the singleton is in whichever of $A[1..m]$ or $A[m+1..n]$ has an odd number of positions. Thus, with two comparisons (one to compare $A[m]$ and $A[m+1]$ and one to determine which of the relevant subarrays has an odd number of positions) we narrow down the search for the singleton to half of the array. Proceeding recursively in this binary-search-like fashion we will find the singleton in $O(\log n)$ comparisons.

Suppose $A[\ell..r]$ is a subarray of A that has odd length (which means that $r-\ell$ is even) and contains the singleton of A. Then the divide-and-conquer algorithm FindSingleton(A, ℓ, r) shown below returns the singleton contained in $A[\ell..r]$. Thus, to find the singleton of A, it suffices to call FindSingleton$(A, 1, n)$.

```
    FindSingleton(A, ℓ, r)
1   if ℓ = r then return ℓ
2   else
3       m := (ℓ + r) div 2
4       if A[m] = A[m + 1] then
5           if m − ℓ is odd then return FindSingleton(A, ℓ, m − 1)   ▷ A[ℓ..m − 1] has odd length
6           else return FindSingleton(A, m + 2, r)                   ▷ A[m + 2..r] has odd length
7       else    ▷ A[m] ≠ A[m + 1]
8           if m − ℓ is even then return FindSingleton(A, ℓ, m)      ▷ A[ℓ..m] has odd length
9           else return FindSingleton(A, m + 1, r)                   ▷ A[m + 1..r] has odd length
```

Correctness: Termination follows because $r-\ell$ is a natural number that decreases in each recursive call. It remains to show:

> If $A[\ell..r]$ has an odd number of positions and contains the singleton position of A, then FindSingleton(A, ℓ, r) returns that position. (*)

We prove this by induction on the length t of the subarray, i.e., $t = r-\ell+1$. The basis, $t = 1$, i.e., $r-\ell = 0$, is obvious. For the induction step, assume that $(*)$ holds for subarrays of length less than t, and consider a subarray $A[\ell..r]$ of odd length $t > 0$ that contains the singleton position.

Because $r-\ell$ is even, in lines 3-9 (where this quantity is not 0), we have that $\ell < m < r$. Furthermore, if $m-\ell$ is even, and since it is not 0, it is at least 2, so $r-m \geq 2$. Therefore, all of $m-1$, $m+1$. and m_2 are in the range $[r..\ell]$ and the recursive calls are all on subarrays of odd length less than t. It remains to show that these subarrays contain the singleton, as the result then is implied by the induction hypothesis: the recursive call, in each case, is made to a subarray of odd length that contains the singleton. This is shown in the following claims.

Claim 1. *If $A[m] = A[m+1]$, the singleton position is in whichever of $A[\ell..m-1]$ or $A[m+2..r]$ has odd length.*

Proof. Suppose $A[m] = A[m+1]$, i.e., positions m and $m+1$ form a couple. Since $A[\ell..r]$ has odd length, exactly one of $A[\ell..m-1]$ and $A[m+2..r]$ has odd length. If $A[\ell..m-1]$ has odd length (i.e., $m-\ell$ is odd) the singleton position cannot be in $A[m..r]$, for, then all of $A[\ell..m-1]$ would consist of couples which contradicts that it has odd length, so the singleton position must be in $A[\ell, m-1]$. By a similar argument, if $A[m+2..r]$ has odd length, it contains the singleton. □

Claim 2. *If $A[m] \neq A[m+1]$, the singleton position is in whichever of $A[\ell..m]$ or $A[m+1..r]$ has odd length.*

PROOF. Suppose that $A[m] \neq A[m+1]$, i.e., positions m and $m+1$ do not form a couple. Since $A[\ell..r]$ has odd length, exactly one of $A[\ell..m]$ and $A[m+1..r]$ has odd length. If $A[\ell..m]$ has odd length, the singleton cannot be in $A[m+1..r]$, for, then $A[m+1..r]$ would have odd length — containing the singleton and some number of couples (this is because position $m+1$, containing a different element than position m, $A[m+1]$, is either the singleton or the first position of a couple) — contradicting that it has even length. So, in this case, the singleton is in $A[\ell..m]$.

Similarly, if $A[m+1..\ell]$ has odd length, it contains the singleton. □

Side note: Claims 1 and 2 hold for any consecutive positions m and $m+1$ in the array; they do not require that they be the "middle" positions. The algorithm works correctly, even if they are not. The choice affects the running time. For example, if t is the length of the subarray and you choose them to be the first and second positions, the resulting algorithm runs in $O(t)$ time; if you choose them to partition the subarray into two pieces of length (roughly) $\sqrt{n}$ and $n - \sqrt{n}$, the resulting algorithm runs in $O(\sqrt{n})$ time. If you choose them to partition the subarray into two pieces of length (roughly) t/c and $t(c-1)/c$, the resulting algorithm runs in $O(\log t)$ time. Choosing the middle two elements minimizes the worst-case number of comparisons, as it minimizes the maximum length of the subarray on which we can recurse.

RUNNING TIME: The running time $T(t)$ of the algorithm, where t is the length of the subarray $A[\ell..r]$, satisfies the recurrence $T(t) = T(\lceil n/2 \rceil) + c$. In terms of the parameters of the Master Theorem we have $a = 1$, $b = 2$, and $d = 0$. Since $a = b^d$, by the Master Theorem $T(t) = \Theta(\log t)$. Since the initial call is to the entire array, which has length n, the running time of the algorithm is $T(n) = \Theta(\log n)$.